

Morelord Core

Module documentation reference

READMEs and user / GM documentation

September 7, 2026 | Local source snapshot

Local manifest version: **0.3.6**. This PDF collects the source documents listed below. Original manual version labels are preserved; older manuals may not cover newer README changes. This conversion does not establish current public release availability.

Included source	Document
morelord-core/README.md	Morelord Core
morelord-core/CORE-GM-PRODUCT-GUIDE.md	Morelord Core: GM Product Guide

Prepared in the shared Morelord document style. Use PDF bookmarks to navigate sections.

Morelord Core

Shared account activation and entitlement services for Morelord Tools Foundry VTT modules.

GM setup

1. Install and enable Morelord Core.
2. Open **Game Settings > Configure Settings > Module Settings > Morelord Core**.
3. Select **Connect or Manage Account**.
4. Approve the temporary code at MorelordGaming.com.

Locations Manager

Saved Associated Scenes remain highlighted when the editor reopens, including when the Scene list is unfocused. Click a Scene to toggle its association, then save the location.

The Settlement Type dropdown includes suggested population ranges: Hamlet (1-100), Village (101-1,000), Town (1,001-6,000), City (6,001-25,000), and Metropolis (25,001+). Road and Other have no population range. These labels provide guidance only; capability tiers remain independent of settlement size.

Export and import settings

GMs can open **Morelord Core > Export / Import Settings** in Module Settings. Export downloads `morelord-settings.json` with configuration from every enabled Morelord module, including hidden defaults and current-browser preferences. Import replaces matching settings and leaves settings absent from the file unchanged. Missing or disabled modules and unknown keys are skipped; enable a module before importing its settings. Reload Foundry after importing to refresh module views.

Account credentials and access caches, shops, shared locations, active journeys, downtime projects, and window positions are excluded. Actor/user references keep their IDs; they are not remapped between worlds. Modules with no configurable settings add no entries. Files are validated before writing; if a write fails, Core attempts to restore the previous values and reports any restoration failures.

Module API

```
const core = game.modules.get("morelord-core")?.api;
await core?.refresh("morelord-marketplace");
if (core?.hasFeature("marketplace.gm-approvals", "morelord-marketplace")) {
  // Enable premium feature.
}
```

Cached entitlements remain usable through the server-provided expiration plus a seven-day offline grace period. Existing world data is never deleted when access expires.

Troubleshooting diagnostics

GMs can select **Download Troubleshooting File** under Troubleshooting in Core's settings list to immediately download a JSON support report. **Connect or Manage Account** contains only account and Core configuration controls. It includes Foundry, game-system, active-module, browser, display, and graphics-renderer versions and capabilities. The sanitized world index is included in the filename to keep reports from different worlds distinct, but not in the JSON. The report excludes account credentials, installation and world identifiers, network addresses, users, and campaign content.

Other Morelord modules can reuse the report builder or start the download through the Core API:

```
const core = game.modules.get("morelord-core")?.api;
const report = core?.getDiagnostics();
core?.exportDiagnostics();
```

Release workflow

Morelord Core uses the same guarded PowerShell release workflow as the other Morelord Foundry modules.

Prerequisites:

- Git and GitHub CLI (gh) installed
- gh auth login completed
- Clean main branch synchronized with origin
- Repository remote pointing to tmoreland72/morelord-core

Validate packaging without changing Git or GitHub:

```
.\release.ps1 -Version 0.1.2 -DryRun
```

Create the release:

```
.\release.ps1 -Version 0.1.2
```

The script updates `module.json`, packages only Foundry runtime files, verifies the ZIP structure and encoding, commits the manifest, creates and pushes an annotated tag, and publishes the GitHub Release.

Foundry manifest URL:

```
https://raw.githubusercontent.com/tmoreland72/morelord-core/main/module.json
```

Anonymous usage statistics

Morelord Core can report only the installed Core version and Foundry version during entitlement refreshes. This is enabled by default and can be disabled by a GM in Module Settings with **Share Anonymous Usage Statistics**. Disabling it does not affect account linking, entitlement checks, or premium access. No campaign, player, actor, item, chat, or world-name data is added to the analytics report.

Standard release workflow

Production Morelord Foundry modules use the same `release.ps1`. Character Export follows these release steps; Downtime remains excluded. Project-specific values are stored in `release.config.json`, so improvements to the workflow can be copied between repositories without editing module logic.

Before a normal release, create `RELEASE-NOTES-x.y.z.md`. The same Markdown file is used for the GitHub Release and parsed into the public Morelord Gaming /releases feed. Recognized headings are Added, Features, Improvements, Changed, Fixed, Breaking Changes, and Security. Prefix a bullet with [Premium] or [Champion] when the change is tier-specific; otherwise it is treated as Standard.

Set the website publishing token once in your PowerShell environment:

```
$env:RELEASE_PUBLISH_TOKEN = "<release publish token>"
```

Validate without changing Git, GitHub, or the website:

```
.\release.ps1 -Version x.y.z -DryRun
```

Publish the normal release:

```
.\release.ps1 -Version x.y.z
```

The normal workflow validates the repository, updates `module.json`, builds and verifies the Foundry ZIP, commits and tags the release, pushes it, creates the GitHub Release from the same release-notes file, and publishes the release to <https://morelordgaming.com/releases>. Draft and prerelease builds intentionally skip the public website feed.

If GitHub release creation succeeds but website publication fails, retry only the idempotent website step:

```
.\release.ps1 -Version x.y.z -WebsiteOnly
```

Use `-SkipWebsitePublish` only when intentionally creating a normal GitHub/Foundry release that should not appear on the Morelord website.

Shared source-book labels

Use `core.sources.resolveBookLabel({ book, custom, pack })` for source filters and displayed book names. Pass both structured source fields and the containing compendium; a legacy string source belongs in `book`. Core resolves configured/localized book labels, strips page references such as PHB Pg. 220, replaces generic pack labels with the owning book, and supplies canonical SRD names. Explicit editions remain distinct. Consumers should display the returned label without title-casing it.

Shared UI services

`core.sources.resolvePackLabel({ pack })` labels source choices using the book, the publisher's declared compendium folder (such as an individual adventure), and the pack title. It reads `manifest.packFolders`, requires no document loading, and leaves the existing `resolveBookLabel` API unchanged. This distinguishes identically named packs within a multi-adventure product, removes title fragments already contained in the book name, and puts compendium types in parentheses.

Core's `ml-card`, `ml-card__body`, and `ml-grid` children constrain their minimum width and wrap long words/identifiers so text cannot widen a card beyond its grid track. Consumers should not add local overflow workarounds for shared cards.

For uniform description cards, use `ml-card` with `data-size="large"` (26rem), a flexible `ml-card__body` preview, optional two-line `ml-card__summary`, and `ml-actions ml-card__footer` for bottom-aligned actions. Add `data-scroll` and `tabindex="0"` to the body to show complete descriptions in a keyboard-accessible scroll region. Without this opt-in, previews remain clipped. The variant is opt-in and does not change existing cards.

Use `ml-grid` with `data-columns="2"` for responsive card collections with Core-owned gaps. Add `data-ml-selectable-card` to an `ml-card` containing one labeled checkbox to make its background and description toggle that checkbox. Core's render hooks activate this behavior automatically for `ml-window` applications; custom renderers can call `core.ui.activateCardSelection(application)`. Links, buttons, labels, disclosure summaries, text selections, and disabled checkboxes retain their native behavior. Keyboard selection remains available through the labeled checkbox. Core owns the hover and checked styling.

`core.ui.renderPreservingScroll(application, renderOperation, options)` preserves page positions by default, plus panels marked with `data-ml-scroll-key`. Pass a per-window positions: `new Map()` to remember keyed panels across tabs. `reset: true` clears saved positions and returns the new page to the top. `selector: "*" , deferred: true` also preserves nested scroll regions after layout settles. Feature modules should use this service instead of maintaining their own capture/restore algorithms.

`core.ui.decorateActorSelect(select, resolveUuid)` displays the selected actor's portrait. UUID-valued selects need no resolver; ID-valued controls can pass `id => game.actors.get(id)?.uuid` while retaining their existing stored values.

`npm run check:design-system` checks all six feature repositories, including Character Export and Downtime. It checks available sources even when a module has no stylesheet directory. Chat templates use the shared chat-card contract rather than the full application shell. Live visual and multiplayer verification remains a separate check.

Accent buttons can opt into `data-variant="outline"`: transparent at rest, orange on hover or keyboard focus. Core page layout preserves intentional `ml-card__body[data-scroll]` regions inside large cards.

Collapsible sections

Use Core's native disclosure component for sections that can be hidden:

```
<details class="ml-surface ml-collapsible-section"
data-ml-section-key="module-id.section-name" open>
  <summary>
    <div class="ml-section-heading"><div><h2>Section title</h2><p>Descriptive
subtitle.</p></div></div>
    <i class="fa-solid fa-chevron-right" aria-hidden="true"></i>
  </summary>
  <div class="ml-collapsible-section__body ml-stack">Section controls and content</div>
</details>
```

Sections start expanded. Core automatically restores and saves their last state per user and world in the current browser, like window geometry. Use a stable, module-prefixed section key, not a journey or document ID. Collapsing only hides the body; it does not reset controls. Native summary activation supports mouse and keyboard. If browser storage is unavailable, toggling still works but cannot persist. Custom renderers can call `MorelordCore.ui.activateCollapsibleSections(application)` after inserting content. Journey Steps is the first consumer.

Dynamic sections can use `core.ui.createCollapsibleSection({ key, title, description, content })`, where `content` is an array of DOM nodes. Append the returned details element, then call `core.ui.activateCollapsibleSections(application)`. Existing controls and their listeners are preserved. Use `dl.ml-quantity-list` with alternating `dt` item names and `dd` quantities for compact, aligned lists inside a shared card.

Character eligibility is centralized in `MorelordCore.ui.participation.listCharacterActors()` and `listCharacterChoices()`: player-owned characters plus character members of the primary party Group, deduplicated. Party members may be Actor documents or stored Actor references. `ownedOnly` further limits the result to characters the current user owns. Modules should use these helpers for character selection and retain their operation-specific permissions.

Core account settings, Ignored Users, and the Location editor use the shared page footer. Save actions remain outside the scrolling content, with native form submission preserved.

Core keeps checkbox descriptions beside their controls, allowing text to wrap within the adjacent column. This applies to checkbox-first and checkbox-last labels, including narrow settings windows; actor choice cards retain their portrait layout.

Settings rows use `label.ml-setting-row` with a `span` containing the label and optional `small` description, followed by the input or select. Core owns responsive control layout and checkbox alignment. Settings templates must use the full Core hero and page footer; the design-system check enforces both.

Morelord Core: GM Product Guide

Morelord Core connects a Foundry VTT world to your Morelord Gaming account. Supported Morelord Tools modules use that connection to check which premium features are available in the world.

This guide is for Game Masters. Players do not need to configure Morelord Core.

What Morelord Core Does

Morelord Core provides one shared account connection for supported Morelord Tools modules. From Foundry, a GM can:

- Connect the current world to a Morelord Gaming account.
- Review the membership level recognized for the world.
- Refresh premium access after an account or membership change.
- Open the Morelord Gaming account page.
- Disconnect the world from the account.

Morelord Core does not remove existing world data if premium access expires.

Requirements

- Foundry Virtual Tabletop version 13.
- Permission to configure the world as a GM.
- A Morelord Gaming account.
- Internet access for the initial connection and periodic access checks.

Install and Enable Morelord Core

1. From Foundry's **Setup** screen, open **Add-on Modules**.
2. Install **Morelord Core**. If you are installing by manifest URL, use:

<https://raw.githubusercontent.com/tmoreland72/morelord-core/main/module.json>

3. Launch the world you want to connect.
4. Open **Game Settings > Manage Modules**.
5. Enable **Morelord Core**, then save the module configuration.

The account connection applies to the current world. Repeat the connection process in each world that should use your Morelord access.

Connect a Morelord Account

1. Open **Game Settings > Configure Settings > Module Settings > Morelord Core**.
2. Select **Connect or Manage Account**.
3. In the **Morelord Account** window, select **Connect Morelord Account**.
4. Foundry opens the Morelord Gaming approval page in your browser and displays a temporary code.

5. Confirm that the code in Foundry matches the prefilled code on the website.
6. On the website, select **Approve Foundry Connection**.
7. Return to Foundry. Foundry checks for approval automatically and reports when the account is connected.

The temporary code expires after 15 minutes. If it expires or activation fails, start the connection process again to create a new code.

If the browser page does not open, select **Reopen Approval Page** in the Foundry window. Check your browser's pop-up settings if the page still does not appear.

Review Account and Access Status

Open **Game Settings > Configure Settings > Module Settings > Morelord Core > Connect or Manage Account**.

When connected, the **Morelord Account** window shows:

- **Installation:** The Foundry world associated with the connection.
- **Membership:** The access level currently recognized for the world.
- **Last validated:** When Foundry last confirmed access with Morelord Gaming.
- **Offline access through:** The expiration time returned by the latest successful access check.

The window also provides three actions:

- **Refresh Access:** Checks Morelord Gaming for current membership and feature access.
- **Open Account:** Opens your Morelord Gaming account page in a browser.
- **Disconnect:** Removes the Morelord account connection and locally cached access information from this world.

Refresh Access

Morelord Core checks access automatically when a connected GM loads the world. Use **Refresh Access** when:

- You changed your Morelord membership.
- You connected or updated a supported Morelord Tools product.
- A premium feature does not reflect a recent account change.
- You want to confirm the most recent validation time.

If Morelord Gaming cannot be reached, Foundry may continue using recently cached access. A warning explains when cached premium access remains available.

Offline Access and Expiration

After a successful access check, Morelord Core stores an access result for use during temporary service or network interruptions. Cached access remains usable through the server-provided expiration time plus a seven-day offline grace period.

After cached access is no longer usable, premium features may be unavailable until Morelord Core successfully validates access again. Existing actors, items, campaign information, and other world data

are not deleted when access expires.

Disconnect a World

1. Open the **Morelord Account** window.
2. Select **Disconnect**.
3. Confirm **Disconnect Morelord Account**.

Disconnecting clears the account token, installation reference, connection label, and cached access information stored by Morelord Core in the current world. It does not delete existing world content. You can reconnect later by completing the approval process again.

Anonymous Usage Statistics

The GM-only **Share Anonymous Usage Statistics** setting is enabled by default. When enabled, Morelord Core includes only these two version values during access checks:

- The installed Morelord Core version.
- The installed Foundry version.

This usage report does not add campaign, player, actor, item, chat, or world-name data. Turning the setting off does not affect account linking, access checks, or premium access.

To change the setting:

1. Open **Game Settings > Configure Settings > Module Settings > Morelord Core**.
2. Enable or disable **Share Anonymous Usage Statistics**.
3. Save your changes.

During the separate account-connection process, Foundry supplies installation and world information needed to label and approve that connection. The anonymous usage-statistics setting applies only to the optional version report sent during access checks.

Troubleshooting

The approval page did not open

Select **Reopen Approval Page**. If nothing happens, allow pop-ups for Foundry or copy the workflow to a browser that can reach MorelordGaming.com.

The temporary code expired

Close or return from the pending activation and start the connection again. Each attempt creates a new temporary code.

Foundry stays on “Waiting for website approval”

Confirm that the code shown in Foundry matches the code on the website and that you selected **Approve Foundry Connection**. If the attempt expires, start again.

A membership change is not showing

Open the **Morelord Account** window and select **Refresh Access**. Confirm the updated value beside **Membership** and check **Last validated** for a recent time.

Morelord Gaming cannot be reached

Confirm that the Foundry host can access <https://morelordgaming.com>. Recently cached access may remain available during the offline grace period. Try **Refresh Access** again after connectivity returns.

Premium access is still unavailable

Confirm that:

- Morelord Core is enabled in the current world.
- The **Morelord Account** window shows **Connected**.
- The expected membership appears in the window.
- **Last validated** reflects a recent successful check.
- The supported Morelord Tools module is installed and enabled.

If needed, disconnect and reconnect the world to create a fresh account connection.

Quick Reference

Goal	Where to go	Action
Connect an account	Morelord Core module settings	Connect or Manage Account > Connect Morelord Account
Check membership	Morelord Core module settings	Open Connect or Manage Account
Update access	Morelord Account window	Refresh Access
Manage the web account	Morelord Account window	Open Account
Stop version reporting	Morelord Core module settings	Disable Share Anonymous Usage Statistics
Remove the connection	Morelord Account window	Disconnect